Bank Layout Optimization for Zero-Skipping FPGA CNNs

Lee Yoo Joon
School of Computer Science and Engineering, Yeungnam University
Information and Communication Engineering Major
Gyeongsan, Republic of Korea
(e-mail)

*Gyu Sang Choi
School of Computer Science and Engineering, Yeungnam University
Information and Communication Engineering Major
Gyeongsan, Republic of Korea
(e-mail)

Abstract—Zero-skipping CNN accelerators often split each convolution window over parallel banks that advance in lockstep, so every window waits for the bank holding the most nonzero activations. When the host builds the windows, it also decides which bank each tap goes to, so the imbalance can be reduced in software: an offline layout of the taps changes the cycle count but neither the bitstream nor the integer output. On an integer CNN core for a Zynq XC7Z020 we give a test, from the layer shapes alone, of where any layout can gain, and compare three layouts: a channel relabeling fitted to activation statistics and folded into the weights, a data-free skew that spreads each channel's taps over the banks, and the two combined. In host emulation over 79 test images of a denoising CNN, at four window values per cycle, the skew cuts the core cycles by 9.2%, more than the fitted relabeling, and the combination by 9.9%; this order held on every tested network whose channel count is a multiple of four. On the board, a cycle model fixed before the runs predicted the cycles of 176 of 180 layers exactly and the other four within one cycle.

Keywords—zero skipping, activation sparsity, load balancing, bank layout, channel permutation, FPGA, CNN accelerator, cycle model

I. INTRODUCTION

A core that skips zero activations saves multiplications, but banks that advance in lockstep make each window cost as much as the busiest one, a stall Cnvlutin measures as its own category [1]. In our core the slot in which the host places a tap decides the tap's bank (Fig. 1), so this stall can be reduced in software, with the bitstream and the integer output unchanged. Known remedies change the network or add hardware: S2TA bounds the nonzeros per block [2], and Sense rebalances channels at run time, noting that input maps "cannot be trained offline to balance IFM workload" [3]. Offline reorderings mostly target weights: SparTen sorts filters by density [4], and Pool and Yu fold a channel permutation into the preceding layer for N:M pruning [5]. VARA, a ReRAM accelerator, permutes inputs offline from training-set activations so that co-zero ones fill skippable crossbar blocks, read through an index unit at run time [6], and skewed storage gives conflict-free parallel access to a matrix's rows and columns [7]. We found no work that spreads co-active channels over lockstep zero-skipping banks by an exact, folded relabeling.

We contribute (i) a test, from the layer shapes alone, of where a layout can change a layer's cycle count (Section II); (ii) three layouts that keep the bitstream and the integer output unchanged: a folded channel relabeling, a data-free skew and both combined (Section III); (iii) board runs of five layouts on four images, the cycles of conv2–conv10 predicted before them and exact in 176 of 180 layers (V-A); and (iv) two pre-registered ten-network studies (V-B) and a comparison with baselines on the board network over 79 images (V-C).

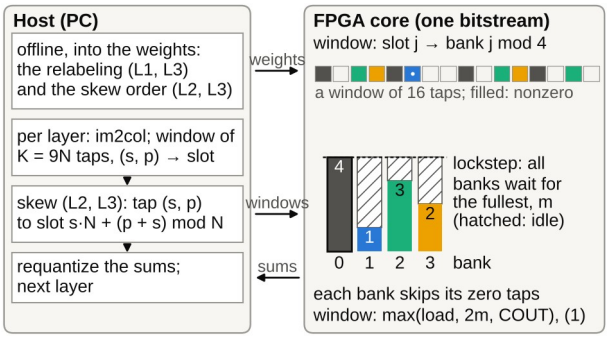


Fig. 1. The system. The slot the host gives a tap decides its bank; the four banks skip zeros in lockstep, so the number m of nonzero taps in the fullest bank sets a window's compute, ⌈COUT/P⌉·m cycles in (1), 2m for 32 outputs. The relabeling and the skew's weight-column order are folded into the weights offline; the host applies the skew's slot order to each window. One bitstream serves every layout.

II. Core, Cost Model and Shape Condition

Core. The host builds each layer's windows (im2col), sends them with the layer's weights and requantizes between layers; the core computes every convolution with INT8 weights, 7-bit unsigned activations, INT32 accumulation, bias and ReLU. Its 64 multipliers form P = 16 output lanes times T = 4 tap banks; its window port takes W values per cycle (W = 2 only in RTL simulation). Slot j of a window belongs to bank j mod 4. In sparse mode, used throughout, each bank stores only its nonzero taps, with their indices in the bank, in one of two window buffers, so the next window loads while one computes; each issue cycle, every bank with a tap left sends one to all P lanes.

Cost of a window. For a window with K taps and COUT outputs, and m the largest number of nonzero taps in one bank, the core spends C cycles:

$$C = \max(\lceil K/W \rceil + 1, \lceil COUT/P \rceil \cdot \max(1, m), COUT), \quad (1)$$

for loading the next window, computing $\lceil COUT/P \rceil$ output groups and emitting COUT results. We call (1) summed over a layer's windows the layer's cycle count, computed in host emulation. On the board each layer also pays a short fill and drain, min($\lceil K/W \rceil$ + 1, the last window's compute) + 3 + g cycles (g: outputs in the last group of P), a rule we read off board runs of four networks on Set12 05.png; conv1 instead takes a measured 17. A layer's count plus its fill and drain gives its predicted cycles.

When a layout can gain. A bank holds at most $\lceil K/T \rceil$ taps. No window is then compute-bound for any input, and no layout can change the count, if

$$\lceil COUT/P \rceil \cdot \lceil K/T \rceil \le \max(\lceil K/W \rceil + 1, COUT). \quad (2)$$

Failing (2) is thus necessary for a gain; without the rounding, (2) reads COUT ≤ P·T/W. So 32-output hidden layers (K = 288: 144 against 145 at W = 2 and 73 at W = 4) can gain only at W = 4, layers with 45 or 64 outputs also at W = 2, and none of these at W = 1; a last layer with COUT ≤ 4 never can. Perfect balance, m = $\lceil n/T \rceil$ for n nonzero taps, bounds every lockstep layout.

Why lockstep. The banks share one issue counter, which runs to the fullest bank's last tap, and each lane sums the four banks' products into one accumulator. Banks with their own queues would need partial sums per bank, lane and window ahead, deeper buffers and a final reduction. Under L0's bank rule they would keep much of the imbalance, which persists over a whole layer: even with unbounded buffers they could save at most 6.5% of L0's count on the board network (a bound from its windows), less than the skew's 9.2% on this core (Table II); with the skew they could beat perfect balance.

III. Three Function-Preserving Layouts

In (ky, kx, channel) order, tap s·N + p (s the 3×3 position, p the channel, N the input channels) is in bank p mod 4 when 4 divides N: in this conventional layout, L0, each bank holds N/4 whole channels (Fig. 2a).

Relabeling (L1). Let π map a position p of a hidden map to its original channel. The producer computes its output channels in the order π, with weight rows, biases and requantization scales reordered, and the consumer's weight column for tap s·N + p comes from channel π(p). As ReLU and requantization act per channel and the consumer adds the same INT32 products, the integer output is identical for every input, while π decides which channels share a bank (Fig. 2b). We permute the outputs of conv1 to conv(L−2) of an L-layer network; the last layer, which can never gain, keeps its input. In the maxout and averaging networks (maximum or mean of maps j and j + N), both maps move with π.

Skew (L2). Skewed storage [7] applied to the taps: when 4 divides N, the host puts channel p of position s in slot s·N + (p + s) mod N, bank (p + s) mod 4, which spreads each channel's nine taps 3/2/2/2 over the banks (Fig. 2c). It permutes the taps of each window and the consumer's weight columns alike, on the inputs of conv2 to conv(L−1) as for L1, and needs no data and no RTL change; no partial sum, in any order, exceeds |bias| + K·127·128 < 2^{31}, so the INT32 result is unchanged.

Skew + relabeling (L3). Under the skew, bank b holds at position s the channels of class (b − s) mod 4 (class: p mod 4): class b at positions 0, 4 and 8, each other class at two. The skew thus dilutes a class's excess of nonzeros without removing it, and which channels form a class is still free: L3 refits the relabeling under this rule, starting from L2, then skews (Fig. 2d).

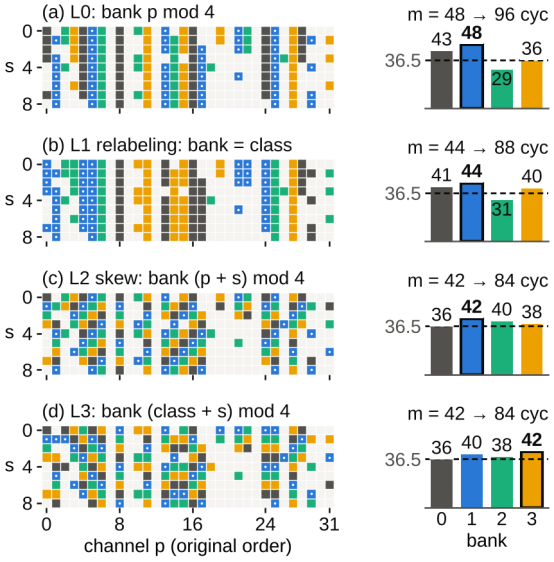


Fig. 2. A conv5 input window of the board network on Set12 05.png (the first with m = 48, the median over the windows compute-bound at W = 4) under L0–L3 (a–d); filled: nonzero, colored by bank (bank 1 dotted). Bars: nonzeros per bank; the fullest, m, costs 2m cycles; dashed: m = 36.5, where 2m equals the load of 73 cycles. Over all of conv5's windows the mean m is 47.6, 42.1, 40.4 and 39.7.

Fitting. Each layer is fitted alone, as relabeled windows are the original's with permuted columns. The recipe draws 2048 windows per layer from one calibration image, Set12 01.png, minimizes their mean of (1) at W = 4, and searches from the identity with 400 proposals, each swapping two channels of

different classes and kept only if the objective falls. Under the conventional bank rule this gives L1, under the skewed rule L3. For the board network we also use L1′, fitted on Set12 01–03 with two noise draws each, from the identity and four random starts (5 × 401 = 2005 objective evaluations per layer).

Function preserved. In host emulation every exported layout reproduced the original's output (every network's L1 on two test images, R1's L1 and L1′ on all 80), and L2 and L3 (L3 up to channel order) every network's INT32 accumulators in every window of Set12 05.png; on the board every layer's output matched the integer model.

IV. Experimental Setup

Networks and images. Ten trained integer DnCNN-style networks [8] for Gaussian noise of σ = 25 on 7-bit inputs: A, 7 layers of 32 channels; B, 10 layers of 64 channels behind a 2×2 space-to-depth input; B-32, the same with 32 channels, with ReLU (R1–R3, training seeds s1–s3), maxout over pairs of 64 computed outputs (M1–M3, seeds s1–s3) or their mean (Av2, seed s2); and B-45, the same with 45 channels and ReLU. All but A had quantization-aware training. The board network, R1, reaches 29.54 dB on Set12 05.png; 37–55% of its hidden layers' window values are nonzero. Test images: Set12 [8] and BSD68 [9], 80 images, one noise draw each; savings are pooled, i.e. computed on cycles summed over the 79 images other than the calibration image, with paired bootstrap 95% intervals over images. Two protocols fixed the recipes, evaluations and hypotheses in advance (Sec. VI).

Board. The W = 4 bitstream for the XC7Z020 closes timing at 100 MHz, uses 11,151 LUTs (21%), 71 of 140 BRAM36-equivalent tiles and no DSP slice, and ran at the board preset's 50 MHz (the cycles do not depend on the clock if the DMA keeps up, which was not measured at 100 MHz). One bitstream ran all layouts; per layer the driver checks the output against the integer model and reads the cycle counter CYCLES and the DMA-wait counter IN_STALL. The five layouts of Table I ran once each on Set12 05–08, with the cycles of conv2–conv10 predicted before the runs (conv1's fill and drain is the measured 17). All other cycle numbers are counts in the host emulation, which reproduces every network's reference PSNR on all 80 images.

V. Results

A. Board Measurement

Table I. Board saving (%) against L0 of R1 at W = 4 on Set12 05–08, one run each (L0 row: total cycles; 08.png is 512×512, the others 256×256). Each run's total equals its predicted cycles, except L0 and L1′ on 08 and L1′ and L3 on 07: one cycle over, in conv8.

Layout	05	06	07	08
L0 (10^6 cycles)	13.289	13.466	13.412	51.977
L1 relabeling	7.02	7.91	7.22	5.93
L1′ relabeling	7.60	9.01	7.89	6.58
L2 skew	8.64	9.97	8.95	7.31
L3 skew + relabeling	9.39	10.68	9.63	7.95

On all four images the board gave L3 < L2 < L1′ < L1 < L0 in cycles, as predicted (Table I); pooled over the four images, L2 saved 8.13% and L3 8.80%. In 176 of the 180 predicted layers (conv2–conv10), CYCLES − IN_STALL equaled the predicted cycles. The other four, all in conv8, were one cycle over; after the runs we found them to be exactly the layers whose last window takes 30 compute cycles (all other conv2–conv9 layers: 32–56), a case absent from the runs the fill-and-drain rule was read off. L2 and L3 had never run on the board before, and 71 of their 72 layers were exact (63 of 64 without conv10, which sees L0's windows). This bitstream's IN_STALL does not count DMA starvation at window boundaries, so a stall common to all layouts could hide in the fill-and-drain rule; the match rules out only stalls that differ between layouts.

B. Ten Networks under Two Protocols

The first protocol's hypotheses held. H1: wherever (2) holds, at W = 1 on all networks and at W = 2 on A and R1–R3, the relabeled count equaled L0's on all 80 images. H2, H3: the pooled saving of L1 was positive on all ten networks at W = 4, 1.09–10.80% (Fig. 3), and on all six where (2) allows it at W = 2, 3.36–7.37% on five and 0.022% on B-45.

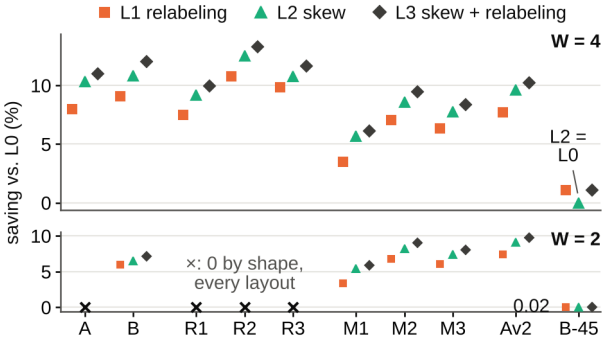


Fig. 3. Pooled saving over the 79 images (count, conv1–convL) at W = 4 (top) and W = 2 (bottom) of L1 (relabeling), L2 (skew) and L3 (skew + relabeling); 95% intervals (±0.39 points at most) lie within the markers. ×: (2) holds in every layer; all layouts count as L0 on all 80 images, as on every network at W = 1. On B-45 (N mod 4 = 1), L0 already has bank (s + p) mod 4, so L2 = L0 and L3 = L1. Networks: Sec. IV (R1: board network).

The second protocol's three also held. H4: wherever (2) holds, L2 and L3 counted exactly as L0 on all 80 images, in every layer. H5: at W = 4, L2 saved more than L1 on all nine networks with N mod 4 = 0, by 0.90–2.36 points. H6: L3 saved more than L2 on all nine, by 0.48–1.18 points, on R1 by 0.73 (95% interval [0.70, 0.75]). Beyond the hypotheses, the gain of L3, fitted on the calibration image alone, is consistent: L3 counted fewer cycles than L2 on all 711 image–network pairs and, pooled, in all 69 fitted layers, and at W = 2 the savings ranked L3 > L2 > L1 on the five networks with N mod 4 = 0 where (2) allows a gain. On these ten networks no image had a larger count under any layout than under L0, at any W. Outside both protocols, on a CIFAR-10 classifier (3×3 convolutions of 32–128 channels, all hidden inputs relaid; fitted on one 32×32 test image), L1, L2 and L3 saved 3.47%, 8.54% and 9.10% at W

= 4 over the other 9,999, L2 and L3 on every image; L1 lost on 176, and at W = 1 all three on some.

C. The Board Network on 79 Images

At W = 4, L2 saved more than both relabelings, and L3 more than L2, on each of the 79 images. The data-free L2 reaches 76.6% of the perfect-balance saving, the most any lockstep layout can save, and L3 82.6%, removing 26% of the imbalance that L2 leaves at the same host cost (Table II). Pooled, L2 also beat the L1 recipe fitted on each test image itself (8.45%) and a Sense-like sort by each channel's number of nonzeros; density balance after SparTen [4] saved least.

D. Cost

Fitting a network took under 30 s (L1 or L3); L2 needs none. Single-threaded on an x86 server, the host's work for 05.png (input, im2col, requantization) took 31–32 ms (timed on the L0 and L1′ exports) against the core's 266 ms at 50 MHz: run back to back, the 7.60% core saving of L1′ is thus 6.3–6.8% end to end, and all of it if the host overlaps the next image; the skew's reordering after im2col adds 4.2 ms. This bound omits the transfer of windows and results; the Zynq's ARM cores would be slower than the server.

TABLE II. R1 AT W = 4 (COUNT, 79 IMAGES): POOLED SAVING AGAINST L0 [95% INTERVAL] AND SHARE (%) OF THE PERFECT-BALANCE SAVING. FIT: IMAGES THE LAYOUT WAS FITTED ON (TEST: EACH TEST IMAGE ITSELF, NO INTERVAL; †: TWO OF THEM AMONG THE 79); DENSITY AND SENSE-LIKE: BEST OF THREE VARIANTS ON THESE IMAGES.

Layout	Fit	Pooled (%)	Share
Density balance [4]	1	3.69 [3.50, 3.88]	30.7
Sense-like sort [3]	test	4.89	40.7
L1 relabeling	1	7.46 [7.16, 7.78]	62.1
L1′ relabeling	3†	8.10 [7.80, 8.39]	67.4
L2 skew	none	9.20 [8.82, 9.58]	76.6
L3 skew + relabeling	1	9.93 [9.54, 10.32]	82.6
Perfect balance	—	12.02 [11.59, 12.44]	100

VI. LIMITATIONS

Scope. One network on the board, four images, one run each; in the counts, ten DnCNN-family denoisers at σ = 25, one noise draw per image, and, outside the protocols, one CIFAR-10 classifier trained for 15 epochs; one fit per layout (the intervals resample images only); BSD68 was part of the networks' development. Banks decoupled by buffers were only bounded (Sec. II), not built; other P, T or kernel sizes and energy were not evaluated.

What was known in advance. L1's saving on R1 was counted before the first protocol, and other relabeling searches had shown a W = 4 saving on B and M1; before the second, L2 had been counted on R1 (all 80 images), on one image of four other networks and on eight images each of B and M1, and L3 on seven images of R1. L1′ was chosen for the board after its BSD68 counts had been seen. L0, L1 and L1′ had run on the board images before: the fill-and-drain rule was read off 05.png runs that include L0's, and the prediction code had to reproduce the 05.png totals of all three, so their 27 layers on 05.png are not out of sample.

VII. CONCLUSION

The layer shapes tell where the tap-to-bank layout of a lockstep zero-skipping core can matter. Where it can, a data-free skew of the taps removed more bank imbalance than a relabeling fitted to activation statistics, on all nine networks with N mod 4 = 0, and a relabeling fitted under the skew removed more still, on every image of every such denoiser. The two only partly add, as the skew already spreads each channel over the banks. On the board the same order held on all four images, and a cycle model fixed before the runs predicted the never-run L2 and L3 to the cycle in 71 of their 72 layers. Code and data: https://github.com/qewr1234/FPGA.

ACKNOWLEDGMENT

This study was conducted with the support of the Ministry of Science and ICT and the Institute for Information & Communications Technology Planning & Evaluation (IITP) under the 'SW-oriented University Program.'

REFERENCES

[1] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. Enright Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 1–13.

[2] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2022, pp. 573–586.

[3] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen, and Y. Kang, "Sense: Model-hardware codesign for accelerating sparse CNNs on systolic arrays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 4, pp. 470–483, 2023.

[4] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2019, pp. 151–165.

[5] J. Pool and C. Yu, "Channel permutations for N:M sparsity," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 13316–13327.

[6] P. Dang, Y. He, and H. Li, "VARA: A voltage-aware ReRAM-based accelerator for energy-efficient computing," arXiv:2609.00421v2, 2026.

[7] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.*, vol. C-20, no. 12, pp. 1566–1569, 1971.

[8] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.

[9] S. Roth and M. J. Black, "Fields of experts," *Int. J. Comput. Vis.*, vol. 82, no. 2, pp. 205–229, 2009.